

Chapter 5 Summary

Summary in Arabic

المخطط الذهني العام للفصل الخامس

(Relational Model) النموذج العلائقي

المفاهيم الأساسية في النموذج العلائقي 1.

- العلاقة (Relation)
- المخطط (Schema)
- الحالة (State)
- السمة (Attribute)
- النطاق (Domain)
- المف/السجل Tuple
- الخصائص الأساسية للعلاقات

في قواعد البيانات العلائقية (Constraints) أنواع القيود 2.

- أ. القيود الضمنية (Inherent/Implicit)
- ب. القيود المعرفة في المخطط (Schema-based/Explicit)
 - قيود النطاق (Domain Constraints)
 - قيود المفتاح (Key Constraints)
 - Superkey
 - Candidate Key
 - Primary Key
 - Entity Integrity (قيود السلامة الكيانية)
 - Referential Integrity (قيود السلامة المرجعية)
- ج. القيود الدلالية (Semantic/Application-based)

(Relational Database Schema) مخطط قاعدة البيانات العلائقية 3.

- تعريف المخطط
- Database State (حالة قاعدة البيانات)
- COMPANY مثال: قاعدة بيانات

عمليات التحديث ومعالجة انتهاكات القيود 4.

- (INSERT, DELETE, UPDATE) عمليات التحديث الأساسية
- أنواع الانتهاكات المحتملة لكل عملية
- (RESTRICT, CASCADE, SET NULL, إلخ) استراتيجيات التعامل مع الانتهاكات

التلخيص التفصيلي

1. المفاهيم الأساسية في النموذج العلائقي

أ. العلاقة (Relation)

- العلاقة هي تمثيل رياضي يشبه جدولاً في قاعدة البيانات.
- كل صف في الجدول يمثل حقيقة عن كيان أو علاقة في العالم الحقيقي.
- مصطلحات رسمية مقابلة للمصطلحات اليومية:

المصطلح غير الرسمي	المصطلح الرسمي
جدول	علاقة (Relation)
عمود (رأس العمود)	سمة (Attribute)
مجموعة القيم الممكنة للعمود	نطاق (Domain)
صف	tuple (سجل)
تعريف الجدول	مخطط العلاقة (Schema)
الجدول المملوء بالبيانات	حالة العلاقة (State)

ب. المخطط (Schema)

- يُكتب على الشكل:
 $R(A_1, A_2, \dots, A_n)$
حيث:
- اسم العلاقة: R .
- السمات (الأعمدة): A_1 إلى A_n .
- مثال:

CUSTOMER(Cust-id, Cust-name, Address, Phone#)

ج. النطاق (Domain)

- هو مجموعة القيم المسموح بها لكل سمة.
- يُحدد نوع البيانات والتنسيق.
- مثال:
- نطاق Cust-id : أرقام مكونة من 6 خانات.
- نطاق #Phone : تنسيق (ddd) ddd-dddd .

💡 ملاحظة: السمة تُعطي "دوراً" للنطاق. فمثلاً، نفس نطاق "التاريخ" يمكن أن يُستخدم لسمتين: Invoice-date و Payment-date ، ولكل منهما معنى مختلف.

د. الـ Tuple (السجل)

- هو مجموعة مرتبة من القيم، كل منها ينتمي لنطاق سمة معينة.

• يُكتب على الشكل: $\langle v_1, v_2, \dots, v_n \rangle$

• مثال:

هو tuple من نوع tuple-4. $\langle \text{"John Smith"}, \text{"101 Main St."}, \text{"(404) 894-2000"}, 632895 \rangle$

هـ. حالة العلاقة (Relation State)

• هي مجموعة من الـ tuples التي تمثل البيانات الفعلية في لحظة زمنية معينة.

• رياضياً:

$$r(R) \subseteq \text{dom}(A_1) \times \text{dom}(A_2) \times \dots \times \text{dom}(A_n)$$

• مثال:

إذا كان $\text{dom}(A_2) = \{a, b, c\}$ و $\text{dom}(A_1) = \{0, 1\}$

فإن الحالة الممكنة: $r(R) = \{\langle 0, a \rangle, \langle 0, b \rangle, \langle 1, c \rangle\}$

و. خصائص العلاقات

1. عدم ترتيب الـ tuples: لا يُعتبر ترتيب الصفوف ذا أهمية.

2. ترتيب السمات والقيم: في التعريف الرسمي، السمات مرتبة، وكذلك القيم داخل كل tuple.

3. القيم الذرية (Atomic): كل قيمة لا يمكن تقسيمها (لا توجد قوائم أو مجموعات داخل خلية).

4. القيمة NULL: تُستخدم للدلالة على:

• قيمة مجهولة

• قيمة غير متاحة

• قيمة غير منطبقة

5. ترميز الوصول للقيم:

- $t[A_i]$ أو $t.A_i \rightarrow$ القيمة في السمة A_i للـ tuple t .
- $t[A_u, A_v] \rightarrow$ مجموعة فرعية من القيم (subtuple).

2. أنواع القيود (Constraints) في قواعد البيانات العلائقية

القيود تضمن صحة البيانات وتماسكها. تنقسم إلى 3 أنواع:

أ. القيود الضمنية (Inherent/Implicit)

• ناتجة من طبيعة النموذج نفسه.

• مثال: لا يُسمح بقيم غير ذرية (مثل القوائم) في أي خلية.

ب. القيود المعرفة في المخطط (Schema-based/Explicit)

وتشمل:

1. قيود النطاق (Domain Constraints)

- كل قيمة في tuple يجب أن تنتمي لنطاق سماتها (أو تكون NULL إذا سُمح بذلك).

2. قيود المفتاح (Key Constraints)

- **Superkey**: tuple مجموعة سمات لا تتكرر قيمتها في أي.
- شرط: $t_1[SK] \neq t_2[SK]$ لأي tupleين مختلفين.
- **Key** (أو **Candidate Key**): Superkey (أي لا يمكن حذف أي سمة منه دون فقدان خاصية التفرد).
- **Primary Key**: يُختار ليكون المفتاح الأساسي (يُشار إليه بخط تحته في المخطط) أحد الـ Candidate Keys.

مثال: 

علاقة CAR(State, Reg#, SerialNo, Make, Model, Year)

- Candidate Keys: {State, Reg#} و {SerialNo}
- إذا اخترنا {SerialNo} كمفتاح أساسي → يُكتب:
CAR(State, Reg#, **SerialNo**, Make, Model, Year)

3. قيود السلامة الكيانية (Entity Integrity)

- لا يُسمح بـ NULL في أي جزء من المفتاح الأساسي.
- لأن المفتاح الأساسي هو وسيلة التعرف على الـ tuple.

4. قيود السلامة المرجعية (Referential Integrity)

- تربط بين علاقتين:
- العلاقة المرجعية (Referencing Relation): تحتوي على مفتاح أجنبي (Foreign Key = FK).
- العلاقة المرجعة (Referenced Relation): تحتوي على المفتاح الأساسي (PK).
- الشرط:
- كل قيمة في FK يجب أن تكون إما:
- 1. موجودة كقيمة PK في العلاقة المرجعة، أو
- 2. NULL — بشرط أن FK المرجعة الأساسي للعلاقة المرجعة FK بشرط أن — NULL.

مثال: 

- علاقة EMPLOYEE(SSN, Name, Dno)
- علاقة DEPARTMENT(Dnumber, Dname, Mgr_ssn)
- هنا: Dno في EMPLOYEE هو FK يشير إلى Dnumber في DEPARTMENT.
- لا يمكن أن يكون Dno = 5 في موظف ما، إلا إذا وُجد قسم برقم 5 في DEPARTMENT.

ج. القيود الدلالية (Semantic/Application-based)

- قيود لا يمكن التعبير عنها بلغة تعريف البيانات (DDL) القياسية.
- مثال: "لا يجوز أن يعمل الموظف أكثر من 56 ساعة أسبوعياً في جميع المشاريع مجتمعة".
- تُنفذ عبر:
- Triggers (مشغلات)
- Assertions (تأكيدات)
- أو في كود التطبيق نفسه.

3. مخطط قاعدة البيانات العلائقية (Relational Database Schema)

- هو مجموعة من مخططات العلاقات + مجموعة القيود بينها.
- يُرمز له بـ:

$$S = \{R_1, R_2, \dots, R_n\} + IC$$

حيث IC = Integrity Constraints.

حالة قاعدة البيانات (Database State)

- هي مجموعة من حالات العلاقات:

$$DB = \{r_1, r_2, \dots, r_n\}$$

- يجب أن تُحقق جميع القيود المعرفة في المخطط.
- إذا لم تُحقق → الحالة باطلة (invalid).

مثال: قاعدة بيانات COMPANY تحتوي على 6 علاقات:

- EMPLOYEE, DEPARTMENT, PROJECT, WORKS_ON, DEPENDENT, DEPT_LOCATIONS

- مع قيود مرجعية مثل:

- EMPLOYEE.Dno → DEPARTMENT.Dnumber
- WORKS_ON.Essn → EMPLOYEE.SSN

4. عمليات التحديث ومعالجة انتهاكات القيود

هناك 3 عمليات أساسية:

أ. INSERT (إدخال سجل جديد)

قد ينتهك:

- قيود النطاق: إذا أدخلت قيمة خارج النطاق.
- قيود المفتاح: إذا تكرر المفتاح.
- السلامة الكيانية: إذا كان PK = NULL.
- السلامة المرجعية: إذا أدخل FK لا يشير إلى PK موجود.

ب. DELETE (حذف سجل)

قد ينتهك السلامة المرجعية فقط:

- إذا كان الـ PK المحذوف مرجعاً من سجلات أخرى.

✅ خيارات المعالجة (يجب تحديدها عند تصميم FK):

- **RESTRICT / REJECT:** رفض الحذف.
- **CASCADE:** حذف جميع السجلات المرتبطة تلقائيًا.
- **SET NULL:** = NULL في السجلات المرتبطة FK جعل قيم.

ج. UPDATE (تعديل سجل)

قد ينتهك:

- قيود النطاق أو **NOT NULL**: إذا عُدلت سمة بقيمة غير صالحة.
- السلامة المرجعية: إذا عُدل FK إلى قيمة غير موجودة.
- المفتاح الأساسي: يُعامل كـ **DELETE + INSERT**، لذا نفس خيارات الحذف تنطبق.

💡 ملاحظة مهمة:

عند انتهاك قيد، يمكن للنظام:

- إلغاء العملية (الأكثر شيوعًا)
- تنفيذ تصحيح تلقائي (مثل CASCADE)
- تنفيذ إجراء مخصص (User-defined routine)

✅ الخلاصة النهائية

النموذج العلائقي يعتمد على العلاقات (الجدول)، وكل علاقة مكونة من سمات ذات نطاقات محددة، وتحتوي على سجلات (tuples) ذرية.

لضمان صحة البيانات، يُطبّق النظام مجموعة من القيود:

- المفتاح لضمان التفرد،
- السلامة الكيانية لمنع NULL في المفتاح،
- السلامة المرجعية للحفاظ على الروابط بين الجداول،
- بالإضافة إلى قيود نطاق وقيود دلالية.

عند تنفيذ عمليات التحديث، يجب التحقق من هذه القيود، واتخاذ الإجراء المناسب عند الانتهاك.

Summary in English

🧠 Mind Map Overview – Chapter 5: The Relational Data Model and Constraints

└─	1. Core Concepts of the Relational Model
└─	└─ Relation (Table)
└─	└─ Schema vs. State
└─	└─ Attribute (Column)
└─	└─ Domain (Value Set)
└─	└─ Tuple (Row/Record)
└─	└─ Key Characteristics of Relations
└─	2. Types of Integrity Constraints
└─	└─ A. Inherent (Implicit) Constraints
└─	└─ B. Schema-Based (Explicit) Constraints
└─	└─ └─ Domain Constraints
└─	└─ └─ Key Constraints
└─	└─ └─ └─ Superkey
└─	└─ └─ └─ Candidate Key
└─	└─ └─ └─ Primary Key
└─	└─ └─ Entity Integrity Constraint
└─	└─ └─ Referential Integrity Constraint (Foreign Keys)
└─	└─ C. Semantic (Application-Based) Constraints
└─	3. Relational Database Schema & State
└─	└─ Definition of a Relational Database Schema
└─	└─ Database State (Instance/Snapshot)
└─	└─ Example: COMPANY Database Schema
└─	4. Update Operations & Constraint Violation Handling
└─	└─ INSERT, DELETE, UPDATE
└─	└─ Possible Violations per Operation
└─	└─ Strategies: RESTRICT, CASCADE, SET NULL, etc.

Detailed Summary (English)

1. Core Concepts of the Relational Model

► Relation

- A **relation** is a mathematical concept based on **set theory**, introduced by **E.F. Codd (1970)** in his seminal paper *"A Relational Model for Large Shared Data Banks"*.
- Informally, it resembles a **table** with rows and columns.
- Each row represents a **fact** about a real-world entity or relationship.

► Schema vs. State

- **Schema (R)**: The **structure** or definition of a relation.
Notation: $R(A_1, A_2, \dots, A_n)$

Example: CUSTOMER(Cust-id, Cust-name, Address, Phone#)

- **State $r(R)$** : The **actual data** (set of tuples) at a given time.

It is a **subset** of the Cartesian product of all attribute domains:

$$r(R) \subseteq \text{dom}(A_1) \times \text{dom}(A_2) \times \dots \times \text{dom}(A_n)$$

► Attribute

- Each column in a relation has a name (e.g., Cust-name).
- Attributes represent **roles** played by domains in the relation.

► Domain

- A **domain** defines the set of **valid values** for an attribute.
- Includes both **logical meaning** and **format/data type**.
 - Example:
 - Domain: USA_phone_numbers
 - Format: (ddd) ddd-dddd
 - Used for attributes like Phone# , Emergency-Contact , etc.

► Tuple

- A **tuple** is an **ordered list** of attribute values: $\langle v_1, v_2, \dots, v_n \rangle$
- Each value $v_i \in \text{dom}(A_i)$
- Example:
 $\langle 632895, \text{"John Smith"}, \text{"101 Main St."}, \text{"(404) 894-2000"} \rangle$ is a 4-tuple in CUSTOMER .

► Key Characteristics of Relations

1. **Tuples are unordered**: The sequence of rows has no meaning.
2. **Attributes are ordered** in the formal definition (though self-describing representations exist).
3. **All values are atomic**: No lists, sets, or nested structures allowed (1NF compliance).
4. **NULL values** are permitted to represent:
 - Unknown values
 - Inapplicable data
 - Missing information
5. **Tuple notation**:
 - $t[A_i]$ or $t.A_i \rightarrow$ value of attribute A_i in tuple t
 - $t[A_u, A_v] \rightarrow$ subtuple of selected attributes

✓ **Informal vs. Formal Terminology:**

| Informal | Formal |

|-----|-----|
| Table | Relation |
| Row | Tuple |
| Column Header | Attribute |
| Column Values | Domain |
| Table Definition | Schema |
| Populated Table | Relation State |

2. Types of Integrity Constraints

Constraints ensure **data correctness** and **consistency**. Three main types:

A. Inherent (Implicit) Constraints

- Built into the relational model itself.
- Example: **No multivalued or composite attributes**—all values must be atomic.

B. Schema-Based (Explicit) Constraints

1. Domain Constraints

- Every value in a tuple must belong to the domain of its attribute (or be NULL if allowed).

2. Key Constraints

- **Superkey**: A set of attributes that **uniquely identifies** tuples.
Condition: For any two distinct tuples t_1 and t_2 , $t_1[SK] \neq t_2[SK]$.
- **Candidate Key**: A **minimal** superkey (no subset is a superkey).
- **Primary Key**: One candidate key chosen as the **main identifier** (underlined in schema diagrams).

✚ Example:

Relation: CAR(State, Reg#, SerialNo, Make, Model, Year)

- Candidate Keys: {State, Reg#} and {SerialNo}
- If {SerialNo} is chosen as primary key → underlined in schema.

3. Entity Integrity Constraint

- **No NULL values** allowed in **any part** of the primary key.
- Reason: Primary keys are used to **identify and reference** tuples.

4. Referential Integrity Constraint

- Ensures consistency between **referencing** and **referenced** relations.
- Involves:
 - **Foreign Key (FK)** in referencing relation R_1
 - **Primary Key (PK)** in referenced relation R_2
- Rule: Every FK value must be either:
 1. Equal to an existing PK value in R_2 , **or**
 2. **NULL** — **only if FK is not part of R_1 's primary key**

✦ Example:

- `EMPLOYEE(Dno)` references `DEPARTMENT(Dnumber)`
- Cannot assign `Dno = 5` unless a department with `Dnumber = 5` exists.

C. Semantic (Application-Based) Constraints

- Cannot be expressed using standard relational model constructs.
- Examples:
 - “An employee cannot work more than 56 hours per week across all projects.”
 - “A manager must have at least 5 years of experience.”
- Enforced via:
 - **Triggers** (`CREATE TRIGGER`)
 - **Assertions** (`CREATE ASSERTION`)
 - Application-level code

3. Relational Database Schema & State

► Relational Database Schema (S)

- Defined as:

$$S = \{R_1, R_2, \dots, R_n\} + IC$$
 where:
 - R_i = individual relation schemas
 - IC = set of integrity constraints (domain, key, referential, etc.)

► Relational Database State (DB)

- A **snapshot** of the database at a point in time:

$$DB = \{r_1, r_2, \dots, r_n\}$$
 where each r_i is a valid state of relation R_i .
- **Valid state**: All tuples satisfy all constraints in IC .
- **Invalid state**: Violates one or more constraints → rejected by DBMS.

✦ **Example:** The **COMPANY database** includes 6 relations:

- EMPLOYEE , DEPARTMENT , PROJECT , WORKS_ON , DEPENDENT , DEPT_LOCATIONS
- With referential constraints like:
- EMPLOYEE.Dno → DEPARTMENT.Dnumber
 - WORKS_ON.Essn → EMPLOYEE.Ssn

4. Update Operations & Constraint Violation Handling

Three basic operations can modify the database state:

► INSERT

May violate:

- **Domain constraint:** Invalid data type or format
- **Key constraint:** Duplicate primary key
- **Entity integrity:** NULL in primary key
- **Referential integrity:** FK references non-existent PK

► DELETE

May violate:

- **Referential integrity only:** If the deleted tuple's PK is referenced elsewhere

✅ **Handling Options** (defined at FK declaration):

- **RESTRICT / REJECT:** Abort deletion
- **CASCADE:** Delete all referencing tuples automatically
- **SET NULL:** Set FK values in referencing tuples to NULL (if allowed)

► UPDATE

May violate:

- **Domain / NOT NULL:** Invalid or missing value
- **Referential integrity:** If FK is changed to non-existent PK
- **Key constraints:** If PK is updated → treated as **DELETE + INSERT**

⚠ **Special Case – Updating Primary Key:**

- Equivalent to deleting the old tuple and inserting a new one.
- Requires same referential integrity options as DELETE.

► General Strategies for Violations

- Cancel the operation (**default**)
 - Cascade changes automatically
 - Set foreign keys to NULL
 - Execute user-defined error-correction routines
-

✓ Final Summary

The **relational model** represents data as **relations (tables)** composed of **atomic tuples (rows)** over well-defined **domains**. To maintain data integrity, the model enforces several **constraints**:

- **Domain constraints** ensure valid attribute values.
- **Key constraints** guarantee unique identification via **candidate** and **primary keys**.
- **Entity integrity** forbids NULLs in primary keys.
- **Referential integrity** maintains consistency across related tables using **foreign keys**.
- **Semantic constraints**, though not natively supported, can be enforced via triggers or application logic.

During **database updates (INSERT, DELETE, UPDATE)**, the DBMS checks for constraint violations and applies predefined strategies (e.g., RESTRICT, CASCADE) to preserve integrity.

This framework forms the **foundation of modern relational database systems** like Oracle, MySQL, and PostgreSQL.
